Ledgerline interest calculation: record layouts and I/O contract

No wiki: record layouts (copybooks) of the Ledgerline batch application only. CONTRACT.md is the input/output contract for the build.

Pages included:

- `record-layouts.md`
- `CONTRACT.md`

`record-layouts.md`

Record layouts

Copybooks of the Ledgerline batch application, as referenced by CONTRACT.md (data definitions only).

LXV022Y

```
      ****************************************************************
      *    Data-structure for  account entity (RECLN 300)
      ****************************************************************
       01  ACCOUNT-RECORD.
           05  LACC-ID                         PIC 9(11).
           05  LACC-ACTIVE-STATUS              PIC X(01).
           05  LACC-CURR-BAL                   PIC S9(10)V99.
           05  LACC-CREDIT-LIMIT               PIC S9(10)V99.
           05  LACC-CASH-CREDIT-LIMIT          PIC S9(10)V99.
           05  LACC-OPEN-DATE                  PIC X(10).
           05  LACC-EXPIRY-DATE              PIC X(10).
           05  LACC-REISSUE-DATE               PIC X(10).
           05  LACC-CURR-CYC-CREDIT            PIC S9(10)V99.
           05  LACC-CURR-CYC-DEBIT             PIC S9(10)V99.
           05  LACC-ADDR-ZIP                   PIC X(10).
           05  LACC-GROUP-ID                   PIC X(10).
           05  FILLER                          PIC X(178).
      *
      * Ver: Ledgerline_v1.0-15-g27d6c6f-68 Date: 2022-07-19 23:15:59 CDT
      *
```

LXV024Y

```
      ****************************************************************
      *    Data-structure for card xref (RECLN 50)
      ****************************************************************
       01 CRD-LINK-RECORD.
           05  LINK-CRD-NUM                  PIC X(16).
           05  LINK-CLNT-ID                   PIC 9(09).
           05  LINK-LACC-ID                   PIC 9(11).
           05  FILLER                         PIC X(14).
      *
      * Ver: Ledgerline_v1.0-15-g27d6c6f-68 Date: 2022-07-19 23:16:00 CDT
      *
```

LXV026Y

```
      ****************************************************************
      *    Data-structure for transaction category balance (RECLN = 50)
      ****************************************************************
       01  TXN-CAT-BAL-RECORD.
           05  TXN-CAT-KEY.
              10 BCAT-LACC-ID                    PIC 9(11).
              10 BCAT-TYPE-CD                    PIC X(02).
              10 BCAT-CD                         PIC 9(04).
           05  TXN-CAT-BAL                        PIC S9(09)V99.
           05  FILLER                              PIC X(22).
      *
```

```
      * Ver: Ledgerline_v1.0-15-g27d6c6f-68 Date: 2022-07-19 23:16:00 CDT
      *
```

LXV027Y

```
      ****************************************************************
      *    Data-structure for disclosure group (RECLN = 50)
      ****************************************************************
       01  RG-GROUP-RECORD.
           05  RG-GROUP-KEY.
              10 RG-LACC-GROUP-ID                    PIC X(10).
              10 RG-TXN-TYPE-CD                     PIC X(02).
              10 RG-TXN-CAT-CD                      PIC 9(04).
           05  RG-INT-RATE                          PIC S9(04)V99.
           05  FILLER                                PIC X(28).
      *
      * Ver: Ledgerline_v1.0-15-g27d6c6f-68 Date: 2022-07-19 23:16:00 CDT
      *
```

LXV030Y

```
      ****************************************************************
      *    Data-structure for TRANsaction record (RECLN = 350)
      ****************************************************************
       01  TXN-RECORD.
           05  TXN-ID                               PIC X(16).
           05  TXN-TYPE-CD                          PIC X(02).
           05  TXN-CAT-CD                           PIC 9(04).
           05  TXN-SOURCE                           PIC X(10).
           05  TXN-DESC                             PIC X(100).
           05  TXN-AMT                              PIC S9(09)V99.
           05  TXN-MERCHANT-ID                      PIC 9(09).
           05  TXN-MERCHANT-NAME                    PIC X(50).
           05  TXN-MERCHANT-CITY                    PIC X(50).
           05  TXN-MERCHANT-ZIP                     PIC X(10).
           05  TXN-CRD-NUM                         PIC X(16).
           05  TXN-ORIG-TS                          PIC X(26).
           05  TXN-PROC-TS                          PIC X(26).
           05  FILLER                                PIC X(20).
      *
      * Ver: Ledgerline_v1.0-15-g27d6c6f-68 Date: 2022-07-19 23:16:01 CDT
      *
```

Candidate I/O contract

What a migrated implementation must produce so the scorer can compare it with the oracle. The contract fixes **formats only**. It says nothing about behaviour, so it may be handed to a builder without leaking the answers.

Inputs (given per run)

The same files the COBOL chain reads, one record per line, fixed width, layouts as in the copybooks:

File	Copybook	Record
`daytxn.dat`	LXV031Y	350
`ledgacc.dat`	LXV022Y	300
`cardlink.dat`	LXV024Y	50
`balcat.dat`	LXV026Y	50
`rategrp.dat`	LXV027Y	50
`clntrec.dat`, `crdrec.dat`, `txntype.dat`, `txncatg.dat`	LXU021C, LXV023Y, LXV028Y, LXV029Y	500, 150, 60, 60

Run parameters: business date `2022-07-18`, ACCRUE parameter `2022071800`.

Step isolation. A candidate for ACCRUE receives the oracle's POSTDAY outputs as its account and category-balance inputs, so an ACCRUE result is never penalised for a POSTDAY error.

Outputs

One JSON file per output: a list of objects, in any order. Field names and types are fixed. Strings may omit trailing spaces. Amounts are JSON strings or numbers holding at most two decimals (`"14.56"`, `-500`).

File	Step	Fields
`postday_txnfile.json`	POSTDAY	transaction fields (below) for every posted transaction
`postday_dayrej.json`	POSTDAY	transaction fields + `reason` (int) + `reason_desc` for every rejected transaction
`postday_ledgacc.json`	POSTDAY	account fields for every account, after posting
`postday_balcat.json`	POSTDAY	`acct_id, type_cd, cat_cd, bal` for every category balance, after posting
`accrue_systxn.json`	ACCRUE	transaction fields for every interest transaction written
`accrue_ledgacc.json`	ACCRUE	account fields for every account, after interest

- Transaction fields: `tran_id, type_cd, cat_cd (int), source, desc, amt, merchant_id (int), merchant_name, merchant_city, merchant_zip, card_num, orig_ts, proc_ts`
- Account fields: `acct_id (int), active_status, curr_bal, credit_limit, cash_credit_limit, open_date, expiration_date, reissue_date, curr_cyc_credit, curr_cyc_debit, addr_zip, group_id`

A run may instead leave the oracle's fixed-width dumps (`<name>.txt`). The scorer reads either form.

Optional `flags.json`: the uncertainties the pipeline declared **before** scoring, as `[{"rule": "R-INT-TRUNC", "source": "<artefact path and line>"}]` or `[{"output": "...", "key": "...", "source": "..."}]`. See PROTOCOL.md §6 for what counts as a flag.